

Project Implementation

For this phase of the project, students have the opportunity to demonstrate mastery of the following SLOs:

- SD2. Create an appropriate database design.
- VV7. Implement a test plan for a medium-size code segment.
- VV10. Undertake, as part of a team activity, a code review of a medium-size code segment.

During each of the 3 sprints for this phase, students have the opportunity to demonstrate proficiency in the following SLOs:

- T1. Follow effective team communication practices.
- T4. Collaborate effectively in cooperative development/programming.
- TE3. Identify configuration items and use a source code control tool in a small team-based project.
- SC1. Write appropriate unit tests for a small component (several functions, a single type, etc.).
- SC2. Write appropriate interface and (if needed) implementation comments for a small component.
- SC4. Select and use a defined coding standard in a small software project.
- SC5. Write implementation documentation for tricky and non-obvious pieces of code.
- SC6. Use test-driven development in coding.
- VV12. Automate testing in a small software project.
- VV13. Use a defect tracking tool to manage software defects in a small software project.

Deliver the following by the end of this phase:

Design your relational database (SD2)

- Store your design in a file in your `docs/` directory of your monorepo.
- In Supabase, build your database.

Project Implementation (VV7, VV10)

- In your server, implement code to access the database using the `supabase.js` library. Put this code in a "database access layer".
- In your server, implement business logic to convert the retrieved data into the format needed to be served via the REST API. Put this code in a "business logic" layer.
- Implement a set of tests to exercise the business logic layer. You probably should mock the database layer so that you have a controlled environment.
- Push an upgraded client prototype to your repo that includes implementations for all the unimplemented user stories (or, at least, the user stories you have time to test and implement).
- Use the Git tagging mechanism to mark this prototype code version (the client/service repos) as **implementation**.
- Note, of course, that any client implementation changes must include unit tests and/or end-to-end tests.